

# 실전반 4회 · 이상 이벤트와 첫 관제 화면

- [이번 회차에서 할 일](#)
- [1. 시작 전 준비 \(복습·환경 확인\)](#)
- [2. 핵심 개념](#)
- [3. 활동 1 — 정상 위상 기준과 이동 통계 적용 · 50분](#)
- [4. 활동 2 — 지속 조건과 이벤트 중복 억제 · 50분](#)
- [5. 활동 3 — 이벤트를 저장하고 제공 화면에 표시 · 50분](#)
- [6. 활동 4 — 오탐 평가 또는 제로샷 예측 비교 후 체크포인트 저장 · 50분](#)
- [7. 실습 과제](#)
- [8. 완료 확인 체크리스트](#)
- [9. 회고 \(10분\)](#)
- [감사 노트](#)
- [용어](#)

| 날짜·시간 | 2026-11-28 (토) 14:00~18:00  | 모듈  | M2 · 원 일정 7번          |
|-------|-----------------------------|-----|-----------------------|
| 블록    | 구현 B5 · B9 / 확장 B2 · B3     | 산출물 | B1→B2→B3→B5→B9 감지 시스템 |
| 완료 확인 | 단발 오류와 지속 이상을 구분하고 시험 구간 고정 |     |                       |

## 회차별 블록 로드맵 — 하나의 시스템에 매 회차 기능을 더한다

구현 새로 만든다 · 확장 조건·연결을 바꾼다 · 플랫폼 교육용 플랫폼에 연결한다

실전반 4학년 · 9회 × 4시간 (함수·질의·정책 수정과 검증)

|             | B1 | B2 | B3  | B4   | B5 | B6   | B7   | B8   | B9   |
|-------------|----|----|-----|------|----|------|------|------|------|
| 1회<br>10/31 | 구현 | 구현 | 구현  |      |    |      |      |      |      |
| 2회<br>11/07 |    | 확장 | 확장  |      |    |      |      |      |      |
| 3회<br>11/21 |    |    | 확장  | 구현   |    |      |      |      |      |
| 4회<br>11/28 |    | 확장 | 확장  |      | 구현 |      |      | 구현   |      |
| 5회<br>01/25 |    |    |     | 확장   |    | 구현   | 구현   |      |      |
| 6회<br>01/26 |    |    | 플랫폼 | 확장+P |    |      | 확장   |      |      |
| 7회<br>01/27 |    |    |     |      |    | 확장+P | 확장+P | 구현+P |      |
| 8회<br>01/28 |    | 확장 |     |      | 확장 |      |      | 확장   |      |
| 9회<br>01/29 |    |    |     |      |    |      |      |      | 확장+P |

통합반 2·3학년 · 10회 × 4시간 (제공 코드의 매핑·설정·조건 완성)

|              | B1 | B2 | B3  | B4   | B5 | B6   | B7   | B8   | B9   |
|--------------|----|----|-----|------|----|------|------|------|------|
| 1회<br>11/14  | 구현 | 구현 |     |      |    |      |      |      |      |
| 2회<br>01/25  |    | 확장 | 구현  |      |    |      |      |      |      |
| 3회<br>01/26  |    |    |     | 구현   |    |      |      |      |      |
| 4회<br>01/27  |    | 확장 | 확장  |      | 구현 |      |      | 구현   |      |
| 5회<br>01/28  |    |    |     | 확장   |    | 구현   |      |      |      |
| 6회<br>01/29  |    |    |     |      |    |      | 구현   | 구현   |      |
| 7회<br>02/01  |    |    | 플랫폼 | 확장+P |    |      |      |      |      |
| 8회<br>02/02  |    |    |     |      |    | 확장+P | 확장+P | 확장+P |      |
| 9회<br>02/03  |    |    |     |      |    |      |      |      | 확장+P |
| 10회<br>02/04 | 확장 |    |     | 확장   |    |      | 확장   | 확장   |      |

이번 회차가 다루는 블록

## 이번 회차에서 할 일

1~3회차에서 관측이 PostgreSQL 에 쌓이고(B1~B3), 설비·센서·SOP 관계를 그래프로 조회할 수 있게 되었다(B4). 오늘은 그 관측을 읽어 **사건(event)** 을 만드는 B5 와, 그 사건을 사람이 보는 B9 관제 화면을 잇는다. 수치 감지는 규칙과 이동 통계로 한다. UCI 재생 데이터에서는 "같은 위상의 정상 사이클과 비교한 잔차"를, 시뮬레이터 스트림에서는 "유효 온도 60°C 초과 10초 연속"을 쓴다. 두 규칙 모두 **지속 조건**과 **중복 억제**가 없으면 단발 오류가 사건이 되거나 같은 이상이 사건 수십 건으로 쪼개진다. 회차가 끝나면 고정된 시험 구간에서 오탐·누락·지연을 숫자로 말하고, 코드·데이터·설정을 학기말 체크포인트로 남긴다. 이것이 1학기(M1·M2)의 마지막 시간이다.

### 학습 목표

- 정상 위상 기준( **PhaseBaseline** )과 이동평균 잔차로 한 사이클의 이상 여부와 감지 지연을 계산한다.
- 지속 카운터가 **MISSING** · **SPIKE** 단발에는 유지되고 유효 저온·**GAP** 에서만 초기화되도록 감지 함수를 고치고, 시스템 **ThresholdDetector** 와 같은 감지 구간을 내는지 검증한다.
- "설비·유형당 열린 사건 하나" 규칙을 구현하고, DB 부분 유일 인덱스와 관제 API 에서 같은 규칙을 확인한다.
- 고정 시험 구간(시드 2026, 시험 180 사이클)에서 지속 조건만 바꿔 평가하고, 오탐을 줄인 것이 지속 조건인지 임계값인지 구분해 설명한다.

## 1. 시작 전 준비 (복습·환경 확인)

모든 명령은 **lecture/system** 에서 실행한다.

```
cd lecture/system
docker compose ps --format '{{.Name}} {{.Status}}'
curl -s localhost:8800/api/state | python3 -c "import json,sys;
d=json.load(sys.stdin); print(d['mode'], d['run_id'], {a:
(round(s['temp_c'],1), s['load']) for a,s in d['sim'].items()})"
```

예상 출력 (상태 시간·run\_id·온도는 실행마다 다르다):

```
hydops-neo4j Up About an hour
hydops-postgres Up About an hour
platform RUN-b5021e42b2 {'HYD-01': (54.0, 0.8), 'HYD-02': (45.0, 1.0), 'HYD-03':
(45.0, 1.0)}
```

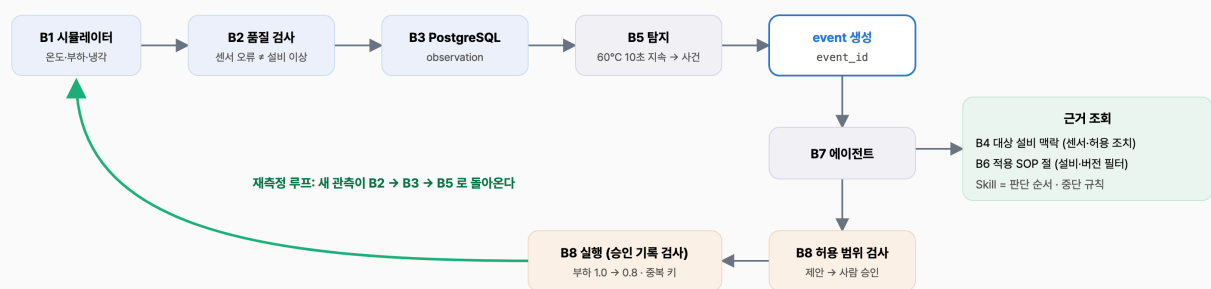
주의: 위 출력은 강사 리허설 서버가 `platform` 모드로 떠 있을 때 얻은 값이다. 수업에서는 강사가 `scripts/servers.sh local 4` 로 띄우므로 `mode` 가 `local` 로 나온다. 이 명령은 학생이 실행하지 않는다(모든 학생의 서버를 재시작한다).

2회차 품질 플래그를 복습한다. `hydops/b2_quality/checks.py` 기준으로 짧은 결측은 `MISSING`, 5초 이상 이어지면 `GAP`, 직전 유효값과 1초 사이 8°C 넘게 뛰면 `SPIKE` (3회 연속이면 수준 변화로 인정), 같은 값 8번째부터 `STUCK` 이다. 정제 값 `value` 는 플래그가 `OK` 일 때만 채워지고, 원시 값 `raw_value` 는 그대로 남는다. 오늘 고칠 첫 버그가 바로 이 두 값을 혼동한 것이다.

## 2. 핵심 개념

### 하나의 사건이 통과하는 경로

종결 기준은 도구 호출 성공이 아니라 조치 이후 새 관측의 회복이다. 모든 기록은 같은 `event_id` 로 이어진다.



B9 관제 같은 `event_id` 로 감지 · 근거(도구 호출-SOP 인용) · 승인 · 명령 결과 · 재측정 판정 을 한 화면에 모은다 — 회복이면 종결, 미개선이면 이관, 센서가 끊기면 판정 보류

### 하나의 사건이 통과하는 경로

오늘 다루는 것은 그림 윗줄 B5 탐지 → event 생성, 그리고 맨 아래 B9 관제 띠다. B7·B8 은 다음 학기에 붙는다. 사건은 `event_id` 를 받는 순간부터 근거·승인·조치·재측정 기록을 모두 같은 키로 모은다. 그래서 사건을 몇 개 만들었는가가 뒤 블록 전체의 부담을 정한다.

### 2.1 두 가지 감지 규칙

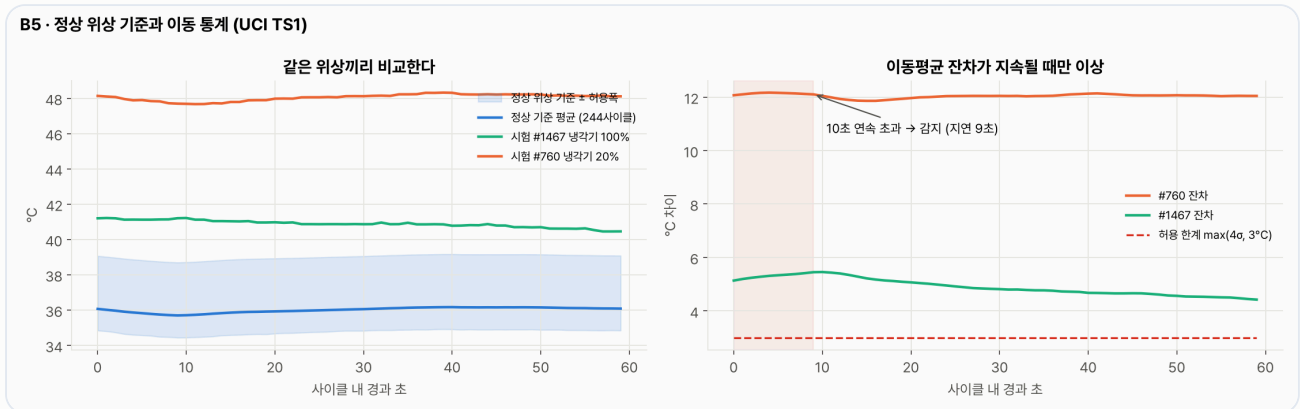
| 데이터    | 함수                                       | 규칙                               | 쓰는 곳                                    |
|--------|------------------------------------------|----------------------------------|-----------------------------------------|
| UCI 재생 | <code>detector.detect_cycle_phase</code> | 잔차 = 5초 이동평균 - 같은 위상 정상 평균. 잔차 > | 오프라인 평가<br>( <code>evaluate.py</code> ) |

| 데이터       | 함수                                           | 규칙                                                                                                                                         | 쓰는 곳                                     |
|-----------|----------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------|
| (사이클 60초) |                                              | $\max(4\sigma, 3^{\circ}\text{C})$ 가 10초 연속이면 이상                                                                                           |                                          |
| 시물레이터 스트림 | <code>detector.ThresholdDetector.feed</code> | 품질 OK 인 온도 > $60^{\circ}\text{C}$ 가 10번 연속이면<br><code>COOLING_ANOMALY</code> ,<br><code>GAP · STUCK</code> 이면<br><code>SENSOR_FAULT</code> | 관제 서버 런타임<br>( <code>runtime.py</code> ) |

두 규칙 모두 **교육용 가정값**이다. UCI 설비의 운전 기준이 아니다. 그리고 둘 다 규칙 기반 감지다. 사전학습 시계열 모델(zero-shot)은 활동 4 의 선택 비교로만 쓴다.

## 2.2 정상 위상 기준

UCI 사이클 안에서 온도는 운전 위상에 따라 조금씩 움직인다. 같은 경과 초( `elapsed_s` )끼리 비교하면 위상 차이를 이상으로 착각하지 않는다.



정상 위상 기준과 이동평균 잔차

왼쪽은 정상 기준 244 사이클의 평균(파랑)과 두 시험 사이클이다. 오른쪽은 잔차다. #760(냉각기 20%)은 잔차가 약  $12^{\circ}\text{C}$  로 한계선(빨간 점선)을 줄곧 넘어 10초째에 감지된다(지연 9초). #1467(냉각기 100%, 정상 라벨)도 잔차가 약  $5^{\circ}\text{C}$  로 한계를 넘는다. 이 사이클이 활동 4 에서 만나는 오탐 1건이다.

주의: `profile.cooler_pct` 는 **설비 상태 라벨**이다. 제품 불량률이 아니다. 그리고 평가에만 쓰

고 탐지 입력에 넣지 않는다.

## 2.3 지속 조건과 중복 억제는 다른 문제를 막는다

| 장치        | 막는 것                       | 코드 위치                                                                                                                      |
|-----------|----------------------------|----------------------------------------------------------------------------------------------------------------------------|
| 지속 조건     | 단발 급등·짧은 흔들림<br>이 사건이 되는 것 | <code>ThresholdDetector.feed</code> 의 <code>above_run</code> ,<br><code>detect_cycle_phase</code> 의 <code>sustain_s</code> |
| 품질 플래그 구분 | 센서 오류 값이 설비 이상으로 세어지는 것    | <code>flag == OK and value is not None</code> 조건                                                                           |
| 중복 억제     | 이미 열린 이상에 사건<br>이 또 생기는 것  | <code>store.create_event</code> 의 <code>open_event</code> + DB 부분 유일 인덱스 <code>ux_event_open</code>                        |

## 3. 활동 1 — 정상 위상 기준과 이동 통계 적용 · 50분

### 목표

고정 분할의 기준 사이클로 `PhaseBaseline` 을 만들고, 두 시험 사이클의 잔차·감지 시점을 직접 계산한다. 한계값을 정하는 항이 실제로는 무엇인지 확인한다.

### 따라 하기

1. 고정 분할을 두 번 불러 같은지 확인하고, 기준을 적합한다.

```
PYTHONPATH=. .venv/bin/python -c "  
from hydops.b5_detect import evaluate as E  
from hydops.b5_detect.detector import PhaseBaseline, detect_cycle_phase  
import numpy as np  
ds, base, test = E.fixed_split(); _, _, test2 = E.fixed_split()  
print(len(base), len(test), test == test2, test[:5])  
temps = ds.sensors['TS1']['mean']; bl = PhaseBaseline.fit(temps, base)  
lim = np.maximum(4*np.asarray(bl.std), 3.0)  
print('mean', round(min(bl.mean),2), round(max(bl.mean),2), 'std',  
      round(min(bl.std),3), round(max(bl.std),3), 'limit', lim.min(),  
      lim.max())  
for cid in (760, 1467):  
    r = detect_cycle_phase(temps[cid], bl)
```

```
print(cid, int(ds.profile.loc[cid, 'cooler_pct']), r['is_anomaly'],
      r['first_alarm_s'], round(r['max_residual'], 2))
"
```

2. 출력의 `limit` 줄을 먼저 읽는다.

## 확인 포인트

```
244 180 True [21, 30, 34, 63, 64]
mean 35.69 36.17 std 0.301 0.323 limit 3.0 3.0
760 20 True 9 12.17
1467 100 True 9 5.46
```

- 기준 244 사이클, 시험 180 사이클이며 두 번 불러도 시험 사이클이 같다( `True` ).
- 위상별 표준편차가 0.30~0.32°C 라서  $4\sigma$  는 약 1.3°C 다. 한계 `max(4σ, 3°C)` 는 모든 위상에서 **3.0°C**, 즉 `min_delta_c` 가 한계를 정한다. `k_sigma` 를 3·4·5 로 바꿔도 결과가 같은 이유다.
- `first_alarm_s` 9 는 "10초 연속" 조건의 10번째 초(0부터 셈)다.

## 흔한 실수

- 기준 사이클에 시험 사이클을 섞는다. `fixed_split` 은 정상·안정 사이클 절반으로 기준을 만들고, **나머지에서만** 상태별 60개를 뽑는다.
- 잔차 계산에 이동평균을 빼먹는다( `smooth_s=1` ). 1초 흔들림이 그대로 한계와 비교된다.
- 시험 결과를 보고 `min_delta_c` 를 고친 뒤 같은 시험 구간으로 다시 보고한다. 그러면 시험 구간이 기준 구간이 된다.

## 4. 활동 2 — 지속 조건과 이벤트 중복 억제 · 50분

### 목표

실습 파일 `labs/practical/S04/detector_lab.py` 의 `SustainDetector.feed` 와 `EventBook.create` 를 고친다. 시작본은 동작은 하지만 조건이 틀렸다.

### 따라 하기

1. 시작본의 핵심 부분을 읽는다.

```
# lecture/labs/practical/S04/detector_lab.py (시작본 발췌)
raw = row.get("raw_value")
if raw is not None and raw > self.threshold_c:
    ...
    st.count += 1
    if st.count == self.sustain_s:
        return [{... "event_type": "COOLING_ANOMALY", ...}]
else:
    st.count = 0
return []
```

2. 시스템의 기준 구현과 비교한다. 유효 관측만 세고, 초기화 조건이 따로 있다.

```
# lecture/system/hydops/b5_detect/detector.py (발췌)
if flag == OK and row["value"] is not None and row["value"] >
    self.th.alarm_temp_c:
    if st.above_run == 0:
        st.above_start = row["ts"]
        st.above_values = []
    st.above_run += 1
    st.above_values.append(row["value"])
    if st.above_run == self.th.alarm_sustain_s:
        ... # Signal(... "COOLING_ANOMALY" ...)
elif flag == OK or flag == GAP:
    st.above_run = 0
# MISSING/SPIKE 단발은 지속 카운터를 유지 (세지도 초기화하지도 않음)
```

3. 같은 시뮬레이터 스트림(시드 7, 30초에 냉각 0.4 주입, 47초에 결측 2초)을 시작본과 정답본에 넣어 본다. `simulate_rows` 는 시뮬레이터 → 품질 검사까지 메모리에서만 돈다.

```
PYTHONPATH=../../labs/practical/S04 .venv/bin/python -c "
import detector_lab as L
rows = L.simulate_rows(150, 0.4, seed=7, degrade_at=30, dropout_at=47,
    dropout_s=2)
print([(r['elapsed_s'], r['raw_value'], r['quality_flag']) for r in rows if
    r['sensor_id']=='TS1' and 42<=r['elapsed_s']<=51])
b = L.EventBook(); print(L.run_stream(rows, L.SustainDetector(), b),
    [(e['window_start'].strftime('%H:%M:%S'),
    e['window_end'].strftime('%H:%M:%S')) for e in b.events])
"
```

4. 두 개의 TODO 를 고친 뒤 같은 명령을 다시 실행한다.

5. 60°C 경계에 머무는 스트림(냉각 0.5 → 정상 상태 온도 30 + 15/0.5 = 60°C)에서 지속 시간을 바꿔 신호 수와 사건 수를 센다.

```
PYTHONPATH=.../labs/practical/S04 .venv/bin/python -c "
import detector_lab as L
rows = L.simulate_rows(300, 0.5, seed=7)
for s in (1, 3, 10):
    print('sustain', s, L.run_stream(rows, L.SustainDetector(sustain_s=s),
    L.EventBook()))
"
```

## 확인 포인트

3단계, 시작본 (48·49초 MISSING 에서 카운터가 끊겨 50초부터 다시 센다):

```
[(42, 58.723, 'OK'), (43, 59.352, 'OK'), (44, 60.683, 'OK'), (45, 61.086, 'OK'),
(46, 61.929, 'OK'), (47, 62.003, 'OK'), (48, None, 'MISSING'), (49, None,
'MISSING'), (50, 63.212, 'OK'), (51, 63.416, 'OK')]
{'signals': 1, 'events_created': 1, 'suppressed': 0} [('14:00:50', '14:00:59')]
```

고친 뒤 (44초부터 유효 10개, 결측 2초는 건너뛰어 55초에 감지):

```
{'signals': 1, 'events_created': 1, 'suppressed': 0} [('14:00:44', '14:00:55')]
```

시스템 `ThresholdDetector` 에 같은 행을 넣으면 (`'COOLING_ANOMALY', '14:00:44', '14:00:55', {... 'valid_samples': 10, 'mean_c': 62.94, 'max_c': 65.05}`) 가 나온다. 정답본과 구간이 같다.

5단계, 시작본(중복 억제 없음)과 정답본:

| 지속  | 신호 | 시작본 사건 | 정답본 사건 | 억제 |
|-----|----|--------|--------|----|
| 1초  | 65 | 65     | 1      | 64 |
| 3초  | 10 | 10     | 1      | 9  |
| 10초 | 0  | 0      | 0      | 0  |

경계에 머무는 온도는 지속 1초에서 신호를 65번 낸다. 지속 조건이 신호를 줄이고, 중복 억제가 남은 신호를 사건 1건으로 묶는다. 지속 10초에서는 300초 동안 유효 값이 60°C 를 103번 넘었지만 10번 연속한 적이 없어 사건이 없다. 이것이 맞는 동작인지는 "경계 운전을 알려야 하는가"라는 운영 질문이다.

## 흔한 실수

- `raw_value` 로 센다. 정상 운전(냉각 1.0) 중 급등 한 번(원시값 74.826, `SPIKE`)만으로 지속 1초 시작본은 신호 1건, 정답본은 0건이다.
- `MISSING` 에서도 초기화한다. 짧은 결측이 잦은 센서에서는 이상이 계속 늦게, 또는 영영 감지되지 않는다.
- `GAP` 에서도 카운터를 유지한다. 5초 이상 모르는 구간을 "계속 높았다"고 가정하는 셈이다.
- 중복 억제를 감지 함수 안의 "한 번 올랐으면 끝" 플래그로 구현한다. 사건이 종결된 뒤 재발해도 다시 울리지 않는다. 억제 기준은 **열린 사건의 상태**여야 한다.

## 5. 활동 3 — 이벤트를 저장하고 제공 화면에 표시 · 50분

### 목표

메모리에서 고친 규칙이 실제 저장소·관제 화면에서 어떻게 지켜지는지 확인한다. 이 활동은 DB 에 쓰지 않고 읽기만 한다.

### 따라 하기

1. 저장 함수를 읽는다. 열린 사건이 있으면 새로 만들지 않고, 동시에 두 요청이 들어와도 DB 인덱스가 막는다.

```
# lecture/system/hydops/b3_tsd/store.py (발췌)
def create_event(run_id, asset_id, event_type, window_start, window_end,
                 rule_version, evidence, status="DETECTED"):
    """사건을 만든다. 같은 설비·유형의 열린 사건이 있으면 새로 만들지 않는다 (중복 억제)."""
    existing = open_event(asset_id, event_type)
    if existing:
        return existing, False
    event_id = f"EVT-{window_start:%Y%m%d-%H%M%S}-{asset_id}-{
        {uuid.uuid4().hex[:4]}"}"
    ...
    except psycopg.errors.UniqueViolation:
        return open_event(asset_id, event_type), False
```

2. 인덱스 정의를 교육용 플랫폼(Lab Platform)의 읽기 전용 질의로 확인한다(6회차에 자세히 쓴다).

```
curl -s -XPOST localhost:8910/fabric/datasources/hydops_edu/query -H 'content-type: application/json' \
```

```
-d "{\"sql\":\"SELECT indexname, indexdef FROM pg_indexes WHERE  
tablename='event'\"}"
```

3. 관제 API 로 사건과 근거 요약을 읽는다. 화면( `static/app.js` )은 1초마다

`/api/state` · `/api/series` · `/api/events` 를 부른다.

```
curl -s "localhost:8800/api/events?limit=3&all_runs=true" | python3 -c "  
import json,sys  
for e in json.load(sys.stdin): print(e['event_id'], e['asset_id'],  
    e['event_type'], e['status'], e['window_start'][11:19], e['window_end']  
    [11:19], e['evidence'])"
```

4. 관제 화면 `http://localhost:8800` 에서 강사가 「단발 급등」 → 「센서 결측」 → 「냉각 성능 저하」 순서로 주입한다. 학생은 사건 목록에 무엇이 생기는지 기록한다.

## 확인 포인트

2단계:

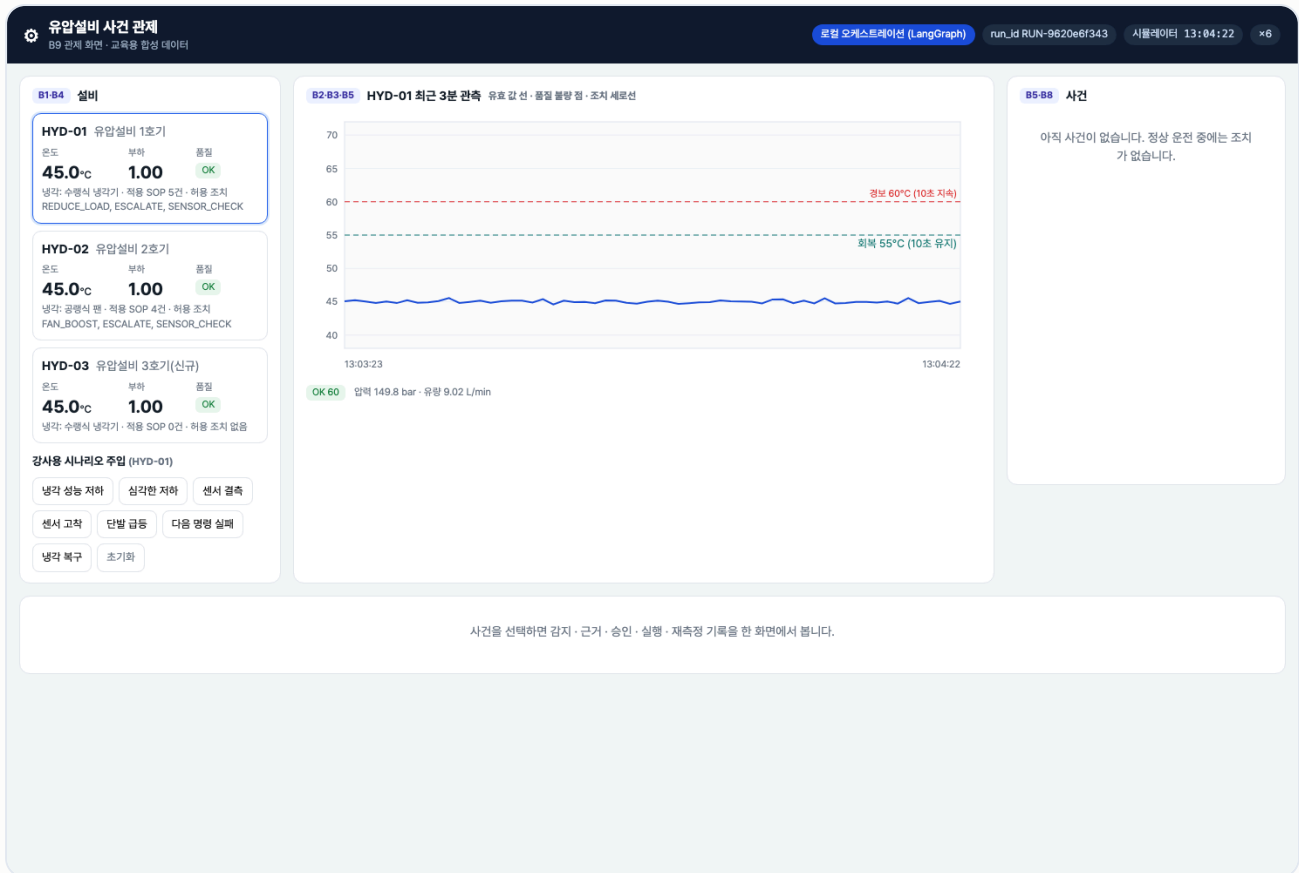
```
{"indexname":"ux_event_open","indexdef":"CREATE UNIQUE INDEX ux_event_open ON  
public.event USING btree (asset_id, event_type) WHERE (status <> ALL  
(ARRAY['CLOSED'::text, 'ESCALATED'::text, 'REJECTED'::text,  
'HOLD_NO_EVIDENCE'::text, 'SENSOR_CHECK'::text, 'RUN_ENDED'::text]))"}
```

`EventBook` 의 `CLOSED_STATUSES` 와 같은 목록이다. 메모리 규칙과 DB 규칙이 어긋나면 한쪽이 억제한 사건을 다른 쪽이 만든다.

3단계 (리허설 DB 의 예, 값은 수업 DB 와 다르다):

```
EVT-20260914-131041-HYD-01-f81e HYD-01 COOLING_ANOMALY CLOSED 13:10:41 13:10:50  
{'rule': 'temp-sustain-v1', 'max_c': 63.91, 'mean_c': 62.09, 'sensor': 'TS1',  
'sustain_s': 10, 'threshold_c': 60.0, 'valid_samples': 10}  
EVT-20260914-130541-HYD-01-61ac HYD-01 COOLING_ANOMALY REJECTED 13:05:41 13:05:50  
{...}  
EVT-20260914-130529-HYD-03-3bf1 HYD-03 COOLING_ANOMALY HOLD_NO_EVIDENCE 13:05:29  
13:05:38 {...}
```

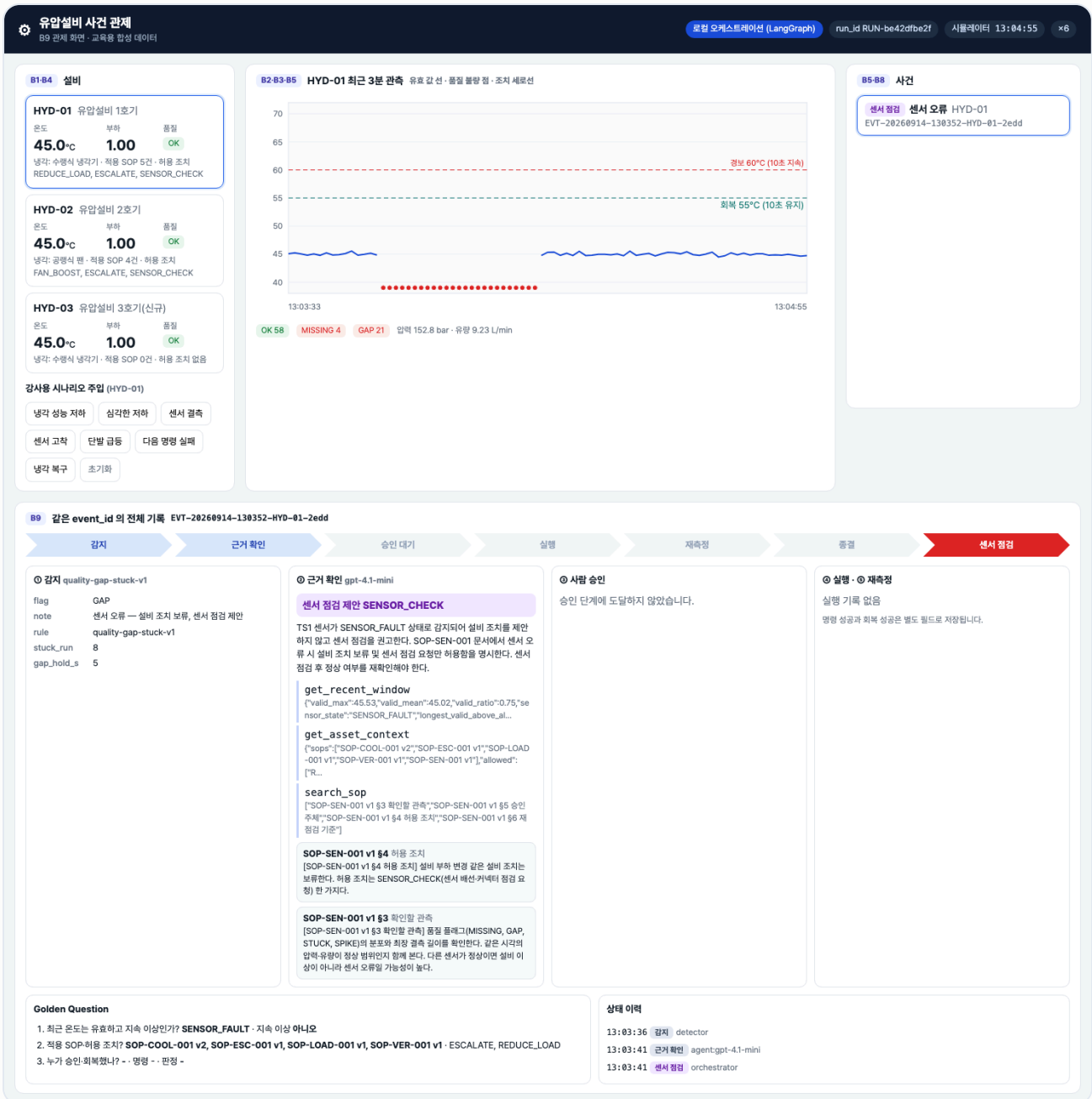
`window_start` 13:10:41 에서 `window_end` 13:10:50 까지 유효 표본 10개다. 근거 요약에 규칙 버전·임계값·지속 시간이 함께 저장되어, 나중에 규칙을 바꿔도 이 사건이 어떤 규칙으로 생겼는지 남는다.



정상 운전 관제 화면

4단계에서 기대하는 화면:

- 단발 급등: 버튼은 20초 주입이고 시뮬레이터는 남은 초가 7의 배수일 때만 +30°C 를 더하므로 7 초 간격으로 급등이 두 번 생긴다. 그래프에 품질 불량 점이 찍히지만 사건 목록은 그대로 비어 있다 (위 S01 상태 유지).
- 센서 결측: **SENSOR\_FAULT** 사건 하나. 아래 화면처럼 **MISSING · GAP** 개수가 표시되고, 설비 조치는 없다.
- 냉각 성능 저하: 60°C 경보선을 넘고 10초 뒤 **COOLING\_ANOMALY** 사건 하나. 온도가 계속 높아 도 두 번째 사건은 생기지 않는다.



센서 결측 → 센서 오류 사건

주의: 오늘 화면의 근거 확인·승인 칸은 5회차 이후 블록(B7·B8)이 채운다. 감사 서버는 완성 시스템이라 칸이 채워져 보일 수 있다. 오늘 확인할 것은 **감지 칸**과 **사건 목록**이다.

## 흔한 실수

- 사건 목록이 비어 있으면 감지 실패로 판단한다. `/api/events` 는 기본으로 **현재 run** 만 준다. 이전 run 은 `all_runs=true` .

- `SENSOR_CHECK` 로 끝난 센서 오류 사건 뒤에 곧바로 냉각 이상 사건이 생긴 것을 중복 억제 실패로 본다. 유형이 다르고, `SENSOR_CHECK` 는 닫힌 상태다.
- 이전 실행에서 열린 채 남은 사건이 새 감지를 막는다. 시스템은 시뮬레이터 초기화 때 `RUN_ENDED` 로 닫는다(회복으로 기록하지 않는다). 학생이 DB 에서 상태를 직접 바꾸지 않는다.

## 6. 활동 4 — 오탐 평가 또는 제로샷 예측 비교 후 체크포인트 저장 · 50분

### 목표

실습 파일의 `fixed_test_ids` 와 `compare_sustain` 을 고치고, 고정 시험 구간에서 지속 조건의 효과를 평가한다. 선택으로 zero-shot 예측과 비교한다. 마지막 10분에 체크포인트를 저장한다.

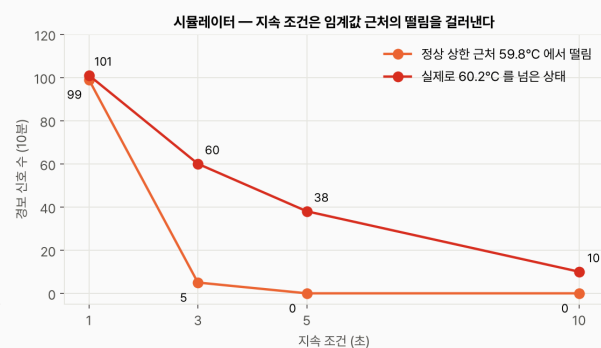
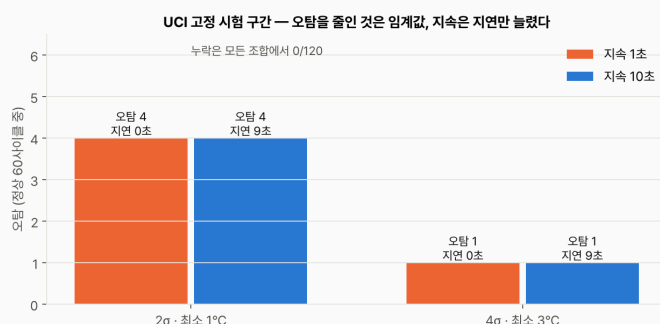
### 따라 하기

1. 제공 평가를 그대로 실행한다.

```
PYTHONPATH=. .venv/bin/python -m hydops.b5_detect.evaluate
```

```
{"params": {"k_sigma": 4.0, "min_delta_c": 3.0, "sustain_s": 10, "smooth_s": 5},
 "baseline_cycles": 244, "test_cycles": 180, "tp": 120, "fp": 1, "fn": 0, "tn": 59, "miss_rate": 0.0, "false_alarm_rate": 0.017, "mean_delay_s": 9.0}
{"params": {"k_sigma": 2.0, "min_delta_c": 1.0, "sustain_s": 1, "smooth_s": 1},
 "baseline_cycles": 244, "test_cycles": 180, "tp": 120, "fp": 4, "fn": 0, "tn": 56, "miss_rate": 0.0, "false_alarm_rate": 0.067, "mean_delay_s": 0.0}
```

#### B5 · 무엇이 오탐을 줄이나: 임계값과 지속 조건은 하는 일이 다르다



임계값과 지속 조건이 하는 일 — UCI 오탐·지연 격자와 임계값 근처 떨림

2. 두 줄은 네 매개변수를 **동시에** 바꿨다. 무엇이 오탐을 줄였는지 한 번에 하나씩 바꿔 확인한다. 시작 본 `compare_sustain` 은 위치 인자가 틀려 `sustain_s` 가 늘 5로 고정된다(모든 행의 지연이 4.0초). 이름 인자로 고친 뒤 실행한다.

```
PYTHONPATH=../../labs/practical/S04 .venv/bin/python -c "
import detector_lab as L
print(L.fixed_test_ids() == L.fixed_test_ids())
for r in L.compare_sustain((1, 5, 10, 20)): print(r)
"
```

3. 임계값 쪽도 한 줄씩 바꿔 본다.

```
PYTHONPATH=. .venv/bin/python -c "
from hydops.b5_detect import evaluate as E
for p in [dict(k_sigma=4.0,min_delta_c=3.0,sustain_s=1,smooth_s=1),
          dict(k_sigma=2.0,min_delta_c=1.0,sustain_s=10,smooth_s=5),
          dict(k_sigma=2.0,min_delta_c=1.0,sustain_s=30,smooth_s=5)]:
    r = E.evaluate(**p); print(p, r['fp'], r['mean_delay_s'], [x['cycle'] for x
        in r['rows'] if x['pred'] and x['cooler_pct']==100])
"
```

4. (선택) zero-shot 예측 비교. Chronos-Bolt tiny 체크포인트는 강사가 미리 내려받아 두었다.

```
HF_HUB_OFFLINE=1 PYTHONPATH=. .venv/bin/python -m hydops.b5_detect.zeroshot
```

5. 체크포인트를 저장한다(아래 확인 포인트의 명령).

## 확인 포인트

2단계, 고친 뒤:

```
True
{'sustain_s': 1, 'tp': 120, 'fp': 1, 'fn': 0, 'tn': 59, 'false_alarm_rate':
0.017, 'miss_rate': 0.0, 'mean_delay_s': 0.0, 'fp_cycles': [1467]}
{'sustain_s': 5, ... 'fp': 1, ... 'mean_delay_s': 4.0, 'fp_cycles': [1467]}
{'sustain_s': 10, ... 'fp': 1, ... 'mean_delay_s': 9.0, 'fp_cycles': [1467]}
{'sustain_s': 20, ... 'fp': 1, ... 'mean_delay_s': 19.0, 'fp_cycles': [1467]}
```

3단계:

```
{'k_sigma': 4.0, 'min_delta_c': 3.0, 'sustain_s': 1, 'smooth_s': 1} 1 0.0 [1467]
{'k_sigma': 2.0, 'min_delta_c': 1.0, 'sustain_s': 10, 'smooth_s': 5} 4 9.0 [1467,
```

```
1471, 1475, 1476]
{'k_sigma': 2.0, 'min_delta_c': 1.0, 'sustain_s': 30, 'smooth_s': 5} 3 29.0
[1467, 1471, 1475]
```

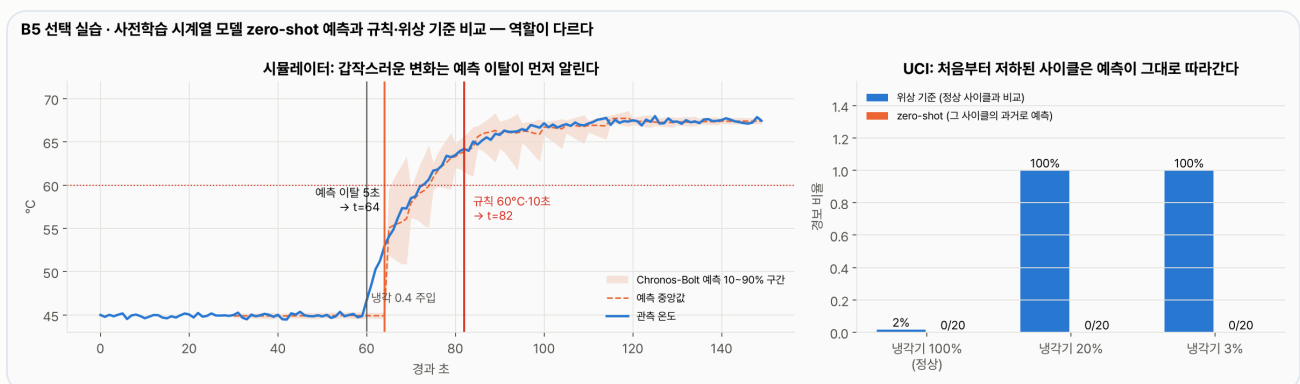
읽는 법:

| 바꾼 것                    | 오탐    | 평균 지연   |
|-------------------------|-------|---------|
| 한계 3°C 고정, 지속 1 → 20초   | 1 → 1 | 0 → 19초 |
| 지속 10초 고정, 한계 1°C → 3°C | 4 → 1 | 9초 유지   |
| 한계 1°C, 지속 10 → 30초     | 4 → 3 | 9 → 29초 |

UCI 시험 구간에서 오탐 4건 → 1건은 **최소 차이 1°C → 3°C** 가 만든 결과다. UCI 사이클은 한 사이클 안에서 냉각기 상태가 바뀌지 않아, 한계를 넘는 사이클은 대부분 60초 내내 넘는다. 그래서 지속 조건은 여기서 지연만 늘린다. 남은 오탐 #1467 은 **stable\_flag=1** (안정 상태 미도달) 정상 사이클이다. 지속 조건의 효과는 활동 2 의 스트림(단발 급등, 60°C 경계)에서 드러난다. **어느 데이터에서 무엇을 막는 지**를 나눠 말하는 것이 오늘의 완료 확인이다.

4단계 (선택):

```
simulator {'inject_s': 60, 'rule_alarm_s': 82, 'zeroshot_alarm_s': 64}
uci {"100": {"cycles": 20, "alarms": 0}, "20": {"cycles": 20, "alarms": 0}, "3": {"cycles": 20, "alarms": 0}}
```



zero-shot 예측과 규칙·위상 기준 비교

시뮬레이터에서 냉각을 60초에 바꾸면 예측 구간 이탈이 64초, 60°C·10초 규칙이 82초에 알린다. UCI 사이클은 처음부터 저하된 상태라 모델이 그 사이클의 과거를 따라가 20·3% 사이클에서도 경보가 0건이다(위상 기준은 100%). 예측 이탈은 "갑자기 달라졌다"를, 위상 기준은 "정상과 다르다"를 본다. 역할이 다르고, 필수 경로는 여전히 규칙과 이동 통계다.

5단계, 체크포인트 저장 (팀 폴더 이름만 바꾼다):

```
CK=../checkpoints/2026-11-28_teamA; mkdir -p $CK
PYTHONPATH=../../labs/practical/S04 CK=$CK .venv/bin/python - <<'EOF'
import hashlib, json, os
from dataclasses import asdict
from datetime import datetime, timezone
from hydops.config import TH, QR
from hydops.b5_detect import evaluate as E
import detector_lab as L
ds, base, test = E.fixed_split()
ck = {"saved_at": datetime.now(timezone.utc).isoformat(timespec="seconds"),
      "split_seed": E.SPLIT_SEED,
      "baseline_cycles": len(base), "test_cycles": len(test),
      "test_ids_sha256": hashlib.sha256(json.dumps([int(x) for x in
      test]).encode()).hexdigest()[:16],
      "thresholds": asdict(TH), "quality_rules": asdict(QR), "sustain_compare":
      L.compare_sustain((1, 5, 10, 20))}
open(os.path.join(os.environ["CK"], "checkpoint.json"), "w").write(json.dumps(ck,
      ensure_ascii=False, indent=2))
print(ck["test_ids_sha256"], ck["baseline_cycles"], ck["test_cycles"],
      [(r["sustain_s"], r["fp"], r["mean_delay_s"]) for r in
      ck["sustain_compare"]])
EOF
(cd .. && tar czf $PWD/$CK/S04_code.tar.gz labs/practical/S04/detector_lab.py)
docker exec hydops-postgres pg_dump -U hydops --data-only -t event -t
      event_history hydops > $CK/events.sql
ls -la $CK
```

```
2bb1593b85bc83a9 244 180 [(1, 1, 0.0), (5, 1, 4.0), (10, 1, 9.0), (20, 1, 19.0)]
```

`test_ids_sha256` 가 `2bb1593b85bc83a9` 로 같으면 2학기에도 같은 시험 구간이다. 리허설에  
서 `checkpoint.json` 1,806 바이트, `events.sql` 26,475 바이트(사건 7행)였다. `pg_dump`  
는 읽기만 한다. 복원은 강사가 5회차 시작 전에 판단한다.

## 흔한 실수

- `fixed_test_ids` 를 고치지 않고 평가한다. 시작본은 부를 때마다 사이클이 달라 `False` 가  
나오고, 오늘과 1월의 결과를 비교할 수 없다.
- F05 그림 제목만 보고 "지속 조건이 오탐을 4건에서 1건으로 줄였다"고 쓴다. 표에서 본 대로 오탐  
을 줄인 것은 최소 차이다.
- zero-shot 결과를 "모델이 이상을 못 찾는다"로만 요약한다. 입력이 그 사이클의 과거뿐이라는 설  
계 조건을 함께 적는다.

- 체크포인트에 `.env` (API 키)를 넣는다. 설정은 `Thresholds` · `QualityRules` 값만 저장한다.

## 7. 실습 과제

- 실전반: `labs/practical/S04/` — `detector_lab.py` 의 `# TODO(학생)` 다섯 곳
  - `SustainDetector.feed` : 유효 값만 세기 · 초기화 조건( `OK` 기준 이하 · `GAP` )
  - `EventBook.create` : 열린 사건 중복 억제와 `suppressed` 증가
  - `fixed_test_ids` : `evaluate.fixed_split()` 의 시험 구간 사용
  - `compare_sustain` : 이름 인자로 `sustain_s` 만 변경
- 검증 방법: 짧은 결측·단발 급등·유효 저온·GAP 단위 사례, 시스템 `ThresholdDetector` 와의 감지 구간 일치, 60°C 경계 스트림의 사건 1건, 시험 구간 고정, `evaluate(sustain_s=...)` 와의 수치 일치.
- 확인 명령: `cd lecture/system && PYTHONPATH=. .venv/bin/python -m pytest ../labs/practical/S04 -q`

시작본 실행 결과:

8 failed, 1 passed in 0.33s

모두 고친 뒤: 9 passed .

## 8. 완료 확인 체크리스트

- ☐ `fixed_split()` 을 두 번 불러 시험 사이클 180개가 같다는 것( `True` )과 `test_ids_sha256` 을 기록했다.
- ☐ 위상 기준의 한계가 모든 위상에서 3.0°C(최소 차이)라는 것을 출력으로 확인했다.
- ☐ 결측 2초가 낀 스트림에서 감지 구간이 14:00:44~14:00:55 로 시스템 감지기와 같다.
- ☐ 정상 운전 중 단발 급등( `SPIKE` )이 지속 1초에서도 신호를 내지 않는다.
- ☐ 60°C 경계 스트림에서 신호가 여러 번이어도 사건은 1건이고 억제 횟수를 말할 수 있다.
- ☐ `ux_event_open` 인덱스의 상태 목록이 `EventBook` 의 닫힌 상태 목록과 같다는 것을 확인했다.
- ☐ 관제 화면에서 단발 급등(사건 없음)·센서 결측( `SENSOR_FAULT` )·냉각 저하( `COOLING_ANOMALY` 1건)를 구분해 기록했다.

- 지속 1·5·10·20초 표에서 오탐은 1건 그대로, 지연만 0→19초로 늘어난다는 것과 그 이유를 적었다.
- `pytest ../labs/practical/S04 -q` 가 9 passed 다.
- 체크포인트 폴더에 `checkpoint.json` ·코드 묶음·사건 덤프가 있다.

## 9. 회고 (10분)

1. 유지보수 담당자가 "경보가 너무 늦다"고 할 때, 지속 10초를 5초로 줄이는 것과 최소 차이 3°C 를 올리는 것은 각각 무엇을 잃는가? 오늘 표의 숫자로 답한다.
2. 60°C 경계 스트림에서 지속 10초는 사건 0건이었다. 온도가 300초 중 103초 동안 60°C 를 넘은 설비를 "정상"으로 뒤도 되는가? 규칙을 바꾸지 않고 이 상황을 드러낼 방법은?
3. 오탐 #1467 은 안정 상태에 도달하지 않은 정상 사이클이다. `stable_flag` 를 탐지 입력에 쓰면 안 되는 이유와, 써도 되는 조건은 무엇인가?

**개인 설명 과제 — 내가 설명할 질의 또는 실패 분기 하나:** 팀원 각자 다음 중 하나를 골라 2분 안에 설명한다. (a) 결측 2초 스트림에서 시작본 감지가 6초 늦어지는 분기(어느 줄에서 카운터가 0이 되는가) · (b) 지속 1초 경계 스트림의 신호 65번이 사건 1건이 되는 경로( `open_event` → `ux_event_open` ) · (c) `compare_sustain` 시작본에서 모든 행의 지연이 4.0초로 같았던 이유 · (d) zero-shot 이 UCI 저하 사이클에서 경보 0건인 이유.

## 강사 노트

### 시간 배분

| 시각          | 내용                                               |
|-------------|--------------------------------------------------|
| 14:00~14:50 | 활동 1 위상 기준 (F04 설명 15분, 실행·한계값 읽기 25분, 질의 10분)   |
| 14:50~15:40 | 활동 2 감지 함수 수정 (시작본 스트림 → 수정 → 경계 스트림 표)          |
| 15:40~16:10 | 휴식 30분                                           |
| 16:10~17:00 | 활동 3 저장·화면 (코드·인덱스 15분, 강사 주입 시연 25분, 기록 정리 10분) |
| 17:00~17:40 | 활동 4 평가 표·선택 zero-shot, pytest                   |
| 17:40~17:50 | 체크포인트 저장                                         |

| 시각          | 내용 |
|-------------|----|
| 17:50~18:00 | 회고 |

## 사전 준비

- `scripts/servers.sh local 4` 로 관제 서버를 띄운다(강사만). 사건 칸이 비어 있는 상태 (S01)에서 시작하려면 수업 직전에 화면의 「초기화」를 한 번 누른다. 이전 run 의 열린 사건은 `RUN_ENDED` 로 닫힌다.
- 활동 3 의 읽기 전용 질의는 교육용 플랫폼의 `hydops_edu` 데이터소스가 필요하다. `scripts/bootstrap_platform.py --upto datasource` 를 미리 실행한다. 플랫폼을 쓰지 않으려면 `psql` 로 같은 SELECT 를 보여 준다.
- zero-shot 선택 실습: `models--amazon--chronos-bolt-tiny` 가 HuggingFace 캐시에 있는지 확인하고 `HF_HUB_OFFLINE=1` 로 실행한다(리허설에서 약 3초).
- 학생 실습은 DB·Neo4j 에 쓰지 않는다. `tests/` 의 시스템 테스트는 `clean` 픽스처가 사건 테이블을 TRUNCATE 하므로 수업 중 공유 DB 에서 실행하지 않는다.

## 장애 대응

- 관제 서버가 멈추면 활동 3 은 S01·S02 스크린샷과 `store.event_trace` 설명으로 대신한다. 활동 2·4 는 서버 없이 진행된다.
- `pg_dump` 가 없는 환경이면 `/api/events?all_runs=true&limit=100` 의 JSON 을 저장한다.
- 학생이 시험 구간 결과로 매개변수를 고르기 시작하면, 기준 구간 안에서만 고르는 방법(기준 사이클을 다시 둘로 나누기)을 안내한다.

## 체크포인트 저장 (학기말)

- 팀별 `checkpoints/2026-11-28_<팀>/` 에 `checkpoint.json` (시드·시험 구간 해시·임계값·품질 기준·지속 비교표), `S04_code.tar.gz` , `events.sql` 을 둔다. 1~3회차 lab 파일도 같은 방식으로 묶는다.
- 5회차(2027-01-25) 환경 복구 때 `test_ids_sha256` 가 `2bb1593b85bc83a9` 인지 가장 먼저 확인한다.

## 용어

| 용어                  | 뜻                                                                                   |
|---------------------|-------------------------------------------------------------------------------------|
| 정상<br>위상<br>기준      | 정상·안정 사이클의 같은 경과 초끼리 구한 평균·표준편차 ( <code>PhaseBaseline</code> )                      |
| 잔차                  | 이동평균한 관측 - 같은 위상의 기준 평균                                                             |
| 지속<br>조건            | 조건이 정해진 횟수만큼 연속으로 참일 때만 이상으로 보는 규칙 ( <code>sustain_s</code> )                       |
| 감지<br>지연            | 이상 조건이 시작된 뒤 경보까지 걸린 초. UCI 평가에서는 <code>first_alarm_s</code>                        |
| 중복<br>억제            | 같은 설비·같은 유형의 열린 사건이 있으면 새 사건을 만들지 않는 것                                              |
| 열린<br>사건            | 상태가<br>CLOSED·ESCALATED·REJECTED·HOLD_NO_EVIDENCE·SENSOR_CHECK·RUN_ENDED<br>가 아닌 사건 |
| 고정<br>시험<br>구간      | 시드 2026 으로 한 번 정한 뒤 바꾸지 않는 평가용 180 사이클 (상태별 60)                                     |
| 오탐 /<br>누락          | 정상 사이클을 이상으로 판정 / 이상 사이클을 놓침                                                        |
| zero-<br>shot<br>예측 | 추가 학습 없이 사전학습 시계열 모델로 다음 구간을 예측하는 것. 규칙 기반 감지와 다르다                                  |
| 체크포<br>인트           | 코드·데이터·설정을 학기 경계에서 복원 가능하게 남긴 묶음                                                    |